Walchand College of Engineering, Sangli

(Government Aided Autonomous Institute)

7CS353 Machine Learning Lab — Assignment 7

Classification: Logistic Regression and Perceptron Learning

Class, Semester	Third Year B. Tech. (CSE), Sem V — AY 2025-26
Linked courses	7CS353 (Lab) and 7CS302 Module IV (Classification)
Builds on	Assignments 5–6 — regression, gradient descent, model evaluation
Structure	Part A: Theoretical concepts (not graded) | Part B: Implementation pipeline (15 marks)
Duration	1 lab session (2 hrs); submit within 1 week

0. Dataset — Student Pass/Fail, 500 Records

The dataset contains 500 student records with two features — study_hours and attendance — and a binary label pass (1 = passed, 0 = failed). It is provided as student_pass.csv. If not available, generate it using the code below.

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
rng = np.random.default_rng(seed=42)
n   = 500
study      = rng.normal(5, 1.8, n).clip(0.5, 10).round(2)
attendance = rng.normal(72, 15, n).clip(30, 100).round(1)
z    = -5 + 0.6*study + 0.04*attendance
prob = 1 / (1 + np.exp(-z))
y    = rng.binomial(1, prob, n)
df   = pd.DataFrame({"study_hours":study,"attendance":attendance,"pass":y})
df.to_csv("student_pass.csv", index=False)
print(df["pass"].value_counts())
```

PART A — Theoretical Concepts

Read before starting Part B. Not graded.

A.1 Binary Classification — Problem Setup

In binary classification, $y \in \{0, 1\}$ and the goal is to learn a function f(x) that assigns the correct class to any new input. Both methods in this assignment produce linear decision boundaries — a hyperplane $w^Tx + b = 0$ that divides the feature space into two regions. The key difference lies in how they find w and b and what they optimise.

A.2 Logistic Regression

Sigmoid Function

Logistic regression models the probability of class 1 by applying the sigmoid (logistic) function to the linear combination $w^Tx + b$:

```
P(y=1 | x)  =  σ(wᵀx + b)  =  1 / (1 + e^(−(wᵀx + b)))
```

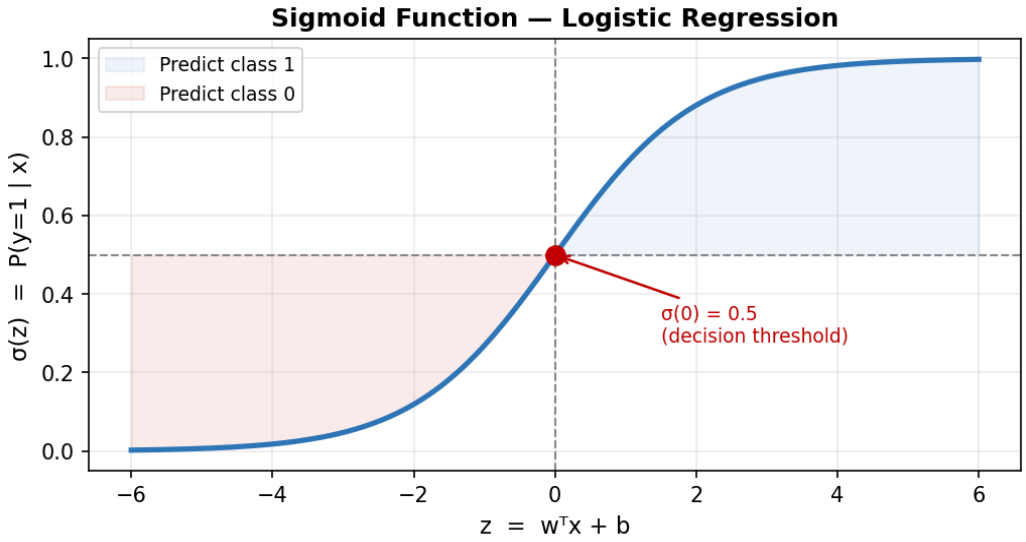


Figure 1: The sigmoid maps any real input to (0,1). The decision boundary is at σ = 0.5 (z = 0). Blue region → predict class 1; red region → predict class 0.

Log-Odds and Coefficient Interpretation

The coefficients can be interpreted through the log-odds: $\log(P(y=1)/P(y=0)) = w^T x + b$. A positive w_j increases the log-odds of class 1 per unit increase in x_j; a negative w_j decreases it.

Loss Function and Training

Logistic regression is trained by minimising Binary Cross-Entropy loss, not SSE:

$$L = -(1/n)\ \Sigma\ [\ y_i\ \log(\hat{y}_i)\ +\ (1-y_i)\ \log(1-\hat{y}_i)\ ]$$

This loss is convex, so gradient descent is guaranteed to reach the global minimum. The update rules at each step are:

$$w \leftarrow w - \alpha \cdot (1/n)\ X^T(\hat{y} - y)$$
$$b \leftarrow b - \alpha \cdot (1/n)\ \Sigma(\hat{y}_i - y_i)$$

$\hat{y}$ here is the sigmoid output, not a linear prediction. This is the key structural difference from linear regression.

A.3 Perceptron Learning Algorithm

The Perceptron Model

The perceptron applies a step function to the weighted sum of inputs, producing a hard binary output:

$$\hat{y} = \text{step}(w^T x + b) = 1 \text{ if } w^T x + b \geq 0, \text{ else } 0$$

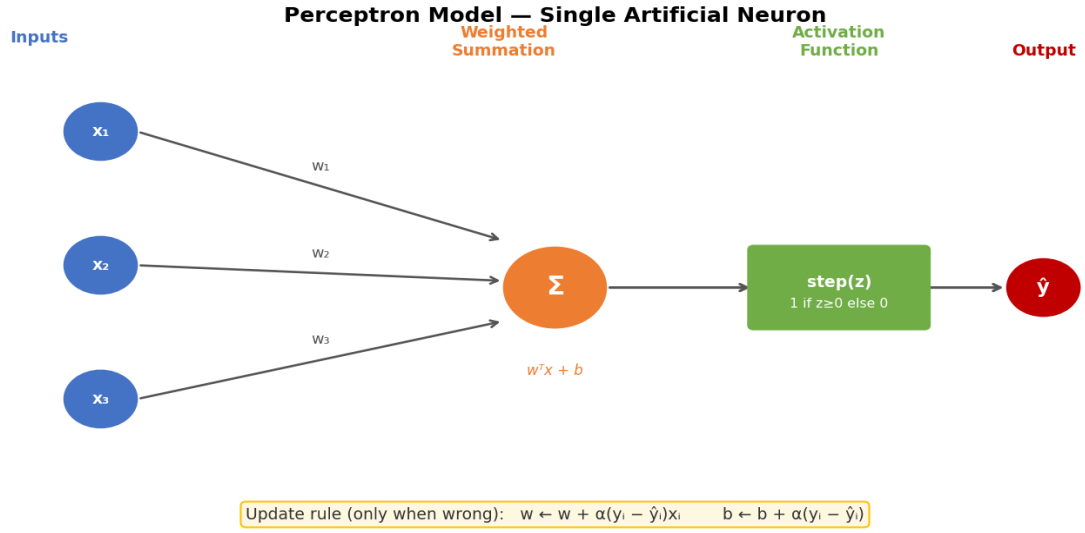


Figure 2: Each input x_j is scaled by a learned weight w_j. The summation node computes $w^Tx + b$. The step activation converts this to a hard 0/1 label. Weights are updated only when the prediction is wrong.

Learning Rule

Unlike logistic regression, the perceptron does not minimise a smooth loss. Weights are updated only on misclassified examples:

```
For each training example (xᵢ, yᵢ):
  ŷᵢ  =  step(wᵀxᵢ + b)
  if ŷᵢ ≠ yᵢ:
    w  ←  w  +  α (yᵢ − ŷᵢ) xᵢ
    b  ←  b  +  α (yᵢ − ŷᵢ)
```

Convergence. If data is linearly separable, the algorithm converges in finite steps. If not, it oscillates — this is the fundamental limitation of the single-layer perceptron. Logistic regression always converges (convex loss) even on non-separable data.

A.4 Evaluation Metrics

Metric	Formula
Accuracy	`(TP + TN) / (TP + TN + FP + FN)`
Precision	`TP / (TP + FP)`
Recall	`TP / (TP + FN)`
F1-Score	`2 · Precision · Recall / (Precision + Recall)`
AUC-ROC	`Area under the ROC curve`

A.5 Comparison

Property	Logistic Regression	Perceptron
Output	Probability ∈ (0,1)	Hard label {0,1}
Activation	Sigmoid σ(z)	Step function
Loss	Binary cross-entropy	No explicit loss — error-driven
Update	Gradient on all data per epoch	Update only on wrong predictions
Convergence	Always (convex loss)	Only if linearly separable
AUC / probability	Yes	No

PART B — Implementation Pipeline (15 marks)

Implement both algorithms from scratch using NumPy. Use sklearn only for verification and evaluation utilities. Every step in its own cell with a one-line markdown comment. All plots must have a title and labelled axes.

B1. Load and explore. Load student_pass.csv. Print shape and class distribution. Draw a scatter plot of study_hours vs attendance coloured by pass. Comment on whether the classes appear linearly separable.

B2. Preprocessing. Perform an 80/20 train-test split (random_state=42) and standardise features.

B3. Implement Logistic Regression from scratch. Plot the loss curve. Print the final w and b, and verify they are close to the coefficients of sklearn LogisticRegression fitted on the same data.

B4. Implement Perceptron from scratch. Track the number of misclassifications per epoch and plot it. Print the final w and b, and verify against sklearn Perceptron.

B5. Evaluate both models. For each model, generate predictions on the test set and produce: (a) a confusion matrix heatmap; (b) a full classification report (precision, recall, F1, accuracy). For logistic regression only, plot the ROC curve and compute AUC. Present a side-by-side markdown summary:

Model	Accuracy	Precision	Recall	F1	AUC
Logistic regression					
Perceptron					

B6. Decision boundaries. Plot the decision boundary of each model over the scatter plot of the original (un-standardised) data, side by side. In a markdown cell, describe any region where the two boundaries differ and explain why.

B7. Interpretation. In a final markdown cell, write a short paragraph (5–7 sentences) addressing: which model performed better and why; what the logistic regression coefficients reveal about the relative importance of study hours vs attendance; why the perceptron cannot produce an AUC score; and in what scenario you would prefer logistic regression over the perceptron.

Deliverables

- One notebook PRN_Assignment7.ipynb — B1–B7 with visible output.
- Both classifiers implemented from scratch using NumPy; sklearn used only for verification.
- All plots with titles and labelled axes.
- Markdown interpretation cells after B3, B4, B5, B6, and B7.
- Submission as per earlier assignments.
- Viva at the start of the next session — be ready to run, modify, and explain any cell live.

Assessment Rubric (15 marks — Part B only, toward LA2)

Assessed against two Course Outcomes. CO2 covers scientific implementation and evaluation of both classifiers; CO3 covers comparison, interpretation, and model selection.

CO	Weight	Rubric statement	Very good	Good	Average	Needs improvement	Not done
CO2	10 marks	Implement logistic regression and perceptron from scratch, train both models, produce evaluation metrics, and visualise decision boundaries.	Both algorithms correctly implemented; loss and misclassification curves plotted; metrics and boundaries complete.	Both implemented; minor errors in update rules or plotting; metrics present.	One algorithm correct; the other incomplete or copied from sklearn.	Neither algorithm implemented; only sklearn used throughout.	Not submitted.
CO3	5 marks	Compare both classifiers scientifically, interpret coefficients and metrics, and justify which model	Precise comparison with evidence; coefficients interpreted correctly; model choice clearly	Comparison present; interpretation mostly correct;	Superficial comparison; limited interpretation of	No comparison or interpretation attempted.	Not done.

		to use for this problem.	justified.	minor gaps in justification.	results.		
Total		**15 marks (CO2 = 10 + CO3 = 5)**					

Implementing an algorithm from scratch and then being unable to explain a line of it in the viva counts as not completed. sklearn-only solutions receive zero for CO2.